

WKND Adventures (masterclass-demo) - Developer Guide

Technical documentation for AEM Edge Delivery Services

June 2, 2026

WKND Adventures (masterclass-demo) - Developer Guide

Technical guide for developers maintaining and extending the WKND Adventures demo site built on AEM Edge Delivery Services.

Quick Reference

Resource	URL
Code Repository	znikolovski/masterclass-demo
Config Service Org	znikolovski
Site Name	masterclass-demo
Preview (main)	https://main-masterclass-demo-znikolovski.aem.page/
Live (main)	https://main-masterclass-demo-znikolovski.aem.live/
Local Dev	http://localhost:3000
Content (DA)	https://content.da.live/znikolovski/masterclass-demo/
Figma Design System	WKND Adventures Design System

Setup type: Standard single-site — one site (masterclass-demo) maps to this repository. No `--pages-url` flag is required for local development.

Architecture Overview

Tech Stack

- Vanilla JavaScript (ES6+ modules, no transpiling)
- CSS3 with custom properties and modern layout (Grid, Flexbox)
- No build step — files served directly from Git via AEM Code Sync
- Content authored in Document Authoring (DA) and delivered at runtime
- Node.js 24 for tooling (lint, traffic simulation, Playwright)
- Configuration via Adobe Config Service (not legacy `helix-config.yaml`)

Project Structure

```
├── blocks/           # 20 content blocks (WKND Adventures palette)
├── templates/        # blog-article, field-notes, landing-page
├── scripts/
│   ├── aem.js        # Core library (DO NOT MODIFY)
│   ├── scripts.js    # Page decoration, martech orchestration
│   ├── delayed.js    # Delayed-phase hook (minimal)
│   └── martech-config.js # Web SDK + Launch URLs
```

		analytics-page.js	# ACDL page context + Analytics mapping
		analytics-acdl.js	# Block interaction events
		plugins/martech/	# aem-martech git subtree (Web SDK + ACDL)
		styles/	
		brand.css	# Design tokens (fonts, colors, spacing)
		styles.css	# Critical global styles
		fonts.css	# Local Roboto fallbacks
		lazy-styles.css	# Below-the-fold global styles
		blog.css	# Blog article template styles
		component-definition.json	# Universal Editor block palette
		component-models.json	# Universal Editor field models
		component-filters.json	# Section/block filters for UE
		tools/scripts/	# Traffic simulator and DA utilities
		ue/	# Universal Editor runtime helpers
		icons/	# SVG icons

Three-Phase Loading (Eager-Lazy-Delayed)

Edge Delivery Services uses a strict Eager-Lazy-Delayed loading strategy to achieve Lighthouse scores of 100. Every file and script must be placed in the correct phase.

Phase	Purpose	What Loads	Performance Impact
Eager	Render above-the-fold content as fast as possible (LCP)	styles/styles.css, first section blocks, scripts/scripts.js	Blocks LCP — keep minimal
Lazy	Load remaining page content after first paint	Remaining header, fonts.css, styles.css, martech lazy hooks	Runs after Eager — safe for non-critical UI
Delayed	Load non-essential third-party scripts	scripts/delayed.js, martech delayed hooks (~3s timeout)	Never blocks rendering

Project-specific E-L-D behavior:

- **Martech (Adobe Analytics + Target):** Loaded via plugins/martech with phased hooks (martechEager, martechLazy, martechDelayed). Target personalization loads eagerly only when page metadata target: on is set; otherwise martech initializes in the lazy phase.
- **Google Fonts:** Instrument Sans and Syncopate load via loadFonts() — deferred until desktop width (≥900px) or after fonts were cached in session storage.
- **Blog template CSS:** styles/blog.css loads lazily when template: blog-article, body class, or /blog/ path is detected.
- **Quick Edit:** Sidekick custom:quick-edit event loads tools/quick-edit/quick-edit.js lazily.

Rules for developers:

- Never add third-party scripts to scripts.js — they block LCP. Use delayed.js or martech delayed hooks.
- Never load fonts eagerly on mobile — loadFonts() gates on viewport width.
- Enable target: on only on pages with active Target experiments (impacts LCP).
- Blocks in the first section load eagerly; all others load lazily automatically.

Key Files and How They Connect

File	Role	Connects To
scripts/aem.js	Core library — loadBlock, loadCSS, decorateBlock, createOptimizedPicture, loadHeader/loadFooter. DO NOT MODIFY.	Imported by scripts.js and blocks
scripts/scripts.js	Entry point — orchestrates E-L-D, buildAutoBlocks, decorateMain, martech init, template/theme classes	Imports aem.js, analytics-page.js, martech-config.js, plugins/martech
scripts/martech-config.js	Web SDK datas-tream/org IDs and Launch embed URLs (staging vs production)	Used by scripts.js martech loader
scripts/analytics-page.js	Builds ACDL page context; maps to Analytics eVars/props via onBeforeEventSend	Called from scripts.js lazy phase
scripts/analytics-acdl.js	Pushes interaction events (carouselChange, faqExpand, etc.)	Imported by interactive blocks
scripts/delayed.js	Placeholder for delayed integrations	Loaded by loadDelayed() after 3s
plugins/martech/src/index.js	aem-martech — Web SDK, ACDL, Target, Launch container loading	Dynamic import from scripts.js
styles/brand.css	CSS custom properties (WKND palette, typography scale)	Imported by styles.css
blocks/{name}/{name}.js	Block decoration — exports default async function decorate(block)	Auto-loaded when block class appears in DOM
component-models.json	Universal Editor field definitions per block	Paired with component-definition.json

Execution flow: page load → scripts.js → loadEager() (decorate main, first section, optional eager martech) → loadLazy() (sections, header/footer, analytics context, martech lazy) → loadDelayed() (martech delayed + delayed.js)

Local Development Setup

Prerequisites

- Node.js 24 (see .github/workflows/main.yaml)
- AEM CLI: `npm install -g @adobe/aem-cli`

Setup Steps

```
git clone https://github.com/znikolovski/masterclass-demo.git
cd masterclass-demo
npm install
aem up
```

Or without global install:

```
npx -y @adobe/aem-cli up --no-open --forward-browser-logs
```

Local Server

- **URL:** `http://localhost:3000`
- Auto-reload on file changes
- Serves code from the working copy (including uncommitted changes)
- Uses preview content from Document Authoring by default

Linting

```
npm run lint      # ESLint + Stylelint
npm run lint:fix  # Auto-fix where possible
```

npm Scripts

Script	Command	Purpose
lint	<code>npm run lint:js && npm run lint:css</code>	Full lint pass (also runs in CI)
lint:fix	ESLint + Stylelint with <code>--fix</code>	Auto-fix style issues
simulate:traffic	<code>simulate-live-audience-traffic.mjs --from-index</code>	Run traffic simulator against production
simulate:traffic:dry	Same with <code>--dry-run</code>	Schedule without browser hits
simulate:traffic:daily	10k hits, concurrency 6, from index	Matches daily GitHub Actions volume
simulate:traffic:backfill	<code>batch-traffic-backfill.mjs</code>	Multi-day backfill for Analytics demo data

Design System

Design tokens live in `styles/brand.css` and are consumed globally via `styles/styles.css`. The Figma Design System is the **visual source of truth** alongside `styles/brand.css` — consult Figma for layout, spacing, and component appearance; use `brand.css` for runtime CSS custom properties.

Figma Variable Collections

Figma variable collections map to CSS custom properties in `styles/brand.css`. When exporting or referencing Figma WEB variables, use the same `var(--token)` names as in code.

Figma Collection	Figma Variables (examples)	CSS Custom Property
Primitives	color/white, color/forest-900, color/orange-500, color/sand-100	Raw hex values consumed by semantic tokens
Semantic colors	background/default, text/primary, link/default, link/hover, accent/primary, surface/light, surface/dark	--background-color, --text-color, --link-color, --link-hover-color, --accent-color, --light-color, --dark-color
Typography	font/heading, font/body, heading/xxl-heading/xs, body/m-body/xs	--heading-font-family, --body-font-family, --heading-font-size-*, --body-font-size-*
Spacing	space/3xl, section/padding, nav/height	--space-3xl, --section-padding, --nav-height

See the Foundations page for the full token reference, color swatches, and type scale.

Design-code Workflow

1. **Before changing a block DOM contract** — consult the matching **hi-fi component** on Blocks — Hi-Fi (block / {block-name} frames) for authoring cell structure, layout, and content model.
2. **For visual specification** — use the block's **Hi-Fi** deep link in the Blocks table below for pixel-accurate layout, spacing, and states.
3. **For shared patterns** — reference Atoms: Button and Atoms: Section for button treatments and section styling conventions.
4. **Implement tokens** — mirror Figma semantic variables as `var(--token)` declarations in `styles/brand.css`; scope block styles to `.{blockname}` selectors in block CSS files.

Figma overview: Blocks — Hi-Fi · Templates — Hi-Fi · Foundations

CSS Custom Properties

Typography

```
--heading-font-family: "Syncopate", sans-serif;
--body-font-family: "Instrument Sans", sans-serif;
--heading-font-size-xxl: 56px;
--heading-font-size-xl: 40px;
--heading-font-size-l: 24px;
--heading-font-size-m: 20px;
--heading-font-size-s: 18px;
--heading-font-size-xs: 14px;
--body-font-size-m: 20px;
--body-font-size-s: 18px;
--body-font-size-xs: 14px;
```

Google Fonts (Instrument Sans, Syncopate) load lazily. Local Roboto variants in `fonts.css` serve as fallbacks.

Colors

```
--background-color: #fff;  
--text-color: #0f1a14;  
--link-color: #0f1a14;  
--link-hover-color: #e8651a;  
--accent-color: #e8651a;  
--light-color: #f4f2ef;  
--dark-color: #0f1a14;
```

Spacing and Layout

```
--space-3xl: 80px;  
--section-padding: 80px;  
--nav-height: 64px;
```

Breakpoints

Name	Min-Width	Usage
Mobile	0	Default styles
Small	768px	@media (width < 768px) overrides
Desktop	900px	@media (width >= 900px) — primary tablet/desktop breakpoint
Large	1200px	Standard Edge Delivery large breakpoint (where used)

The project primarily uses 900px as the desktop breakpoint, matching Edge Delivery Services conventions.

Section Styles

Applied via section-metadata block or Universal Editor section style field:

Class	Effect
inverse / dark	Dark background, inverted text and buttons
secondary	Light beige background
accent	Orange accent section
light / highlight	Subtle highlight treatments
narrow	Constrained content width
hero-adventure-container	Full-bleed hero layout with accent pairing
carousel-blog-wrapper / carousel-blog-container	Carousel blog layout wrappers

Fonts

Family	Source	Usage
Syncopate	Google Fonts (400, 700)	Headings
Instrument Sans	Google Fonts (400–700)	Body copy
Roboto / Roboto Condensed	Local WOFF2 in fonts/	Fallback weights

Martech and Analytics

Architecture

The site integrates Adobe Experience Platform Web SDK and Adobe Target via the aem-martech plugin (plugins/martech/). Configuration is in scripts/martech-config.js:

- **Datastream ID:** 56dee4fc-21a9-4e37-83ab-bdd874957aba
- **Org ID:** 28260E2056581D3B7F000101@AdobeOrg
- **Launch staging:** Loaded on .aem.page and localhost
- **Launch production:** Loaded on .aem.live

See docs/MARTECH.md and docs/ANALYTICS-LAUNCH-PLAN.md for full Launch property setup and variable mapping.

Page Metadata (Universal Editor)

Field	Values	Effect
target	on / off	Eager Target personalization when on
analytics	on / off	Auto page view via Web SDK (default on if unset)
adventureCategory	climbing, trekking, etc.	Audience segment (eVar4); auto-inferred from URL if empty
journeyStage	inspiration, discovery, planning, community	Intent segment (prop3); auto-inferred if empty

ACDL Interaction Events

Blocks push events to window.adobeDataLayer via scripts/analytics-acdl.js:

Event	Block	Trigger
carouselChange	carousel-blog, carousel-hero	Slide index change
faqExpand	accordion-faq	FAQ item opened

Page context is built in scripts/analytics-page.js and pushed in the lazy phase before the Web SDK page view.

Consent

A Phase 1 consent stub grants collect/personalize/marketing on preview (.aem.page), live (.aem.live), and localhost. Wire a CMP and update updateUserConsent() before production hardening.

Blocks Reference

Customized blocks (excluding boilerplate-only defaults):

Block	Purpose	Variants / Notes	Key Features	Figma
accordion-faq	Expandable FAQ list	—	Pushes faqExpand ACDL events; extracts question labels from cell structure	Hi-Fi
cards	Image + text card grid	—	Standard card layout from Block Collection	Hi-Fi
cards-steps	Numbered step cards	—	Step number + content per row	Hi-Fi
carousel	UE alias for carousel-blog	Delegates to carousel-blog	Authors block “Carousel” in UE	Hi-Fi
carousel-blog	Blog post carousel	carousel-blog-wrapper section style	Fetches posts, optimized images, carouselChange analytics	Hi-Fi
carousel-hero	Full-width hero carousel	—	Slide indicators, ACDL slide tracking	Hi-Fi
columns	Multi-column layout	2, 3, 4 columns	Responsive column grid	Hi-Fi
columns-article	Article sidebar layout	—	Main + aside column structure	Hi-Fi
columns-featured	Featured content columns	—	Highlight column variant	Hi-Fi
columns-gallery	Image gallery columns	—	Gallery row/cell model	Hi-Fi
footer	Site footer	—	Navigation, social, copyright	Hi-Fi
fragment	Content fragment inclusion	—	Auto-loads / fragments/ links; Fragment Picker support	Hi-Fi
header	Site header / nav	—	Navigation, brand logo	Hi-Fi
hero	Boilerplate hero (auto-block)	Delegates to hero-adventure when styled	Created by buildHeroBlock() unless hero-adventure present	Hi-Fi
hero-adventure	WKND hero with background image	hero-adventure-content section	Optimized picture (fetchpriority=high), adopts orphan h1/author fields	Hi-Fi
marquee-ticker	Scrolling ticker	—	Animated marquee items	Hi-Fi
section-video-metadata-activity	Section video metadata activity player	inverse, accent, narrow, etc.	Drives section CSS classes via metadata IDs; plays enhanced embed; ACDL play events	Hi-Fi Hi-Fi

Auto-blocking: `buildHeroBlock()` in `scripts.js` creates a hero block from leading h1 + picture unless hero-adventure is already present. Fragment links (`/fragments/`) auto-resolve via the fragment block.

Hi-Fi visual specs: All block treatments live on Blocks — Hi-Fi. Each frame is labeled block / {block-name}.

Hi-Fi mappings: hero shares the hero-adventure hi-fi component; carousel shares the carousel-hero hi-fi component. fragment, youtube-video, and section-metadata link to the Blocks — Hi-Fi overview until dedicated hi-fi components are added.

Universal Editor Component Models

Models are defined at the repository root (not in a `models/` folder):

- `component-definition.json` — Block palette and DA plugin bindings
- `component-models.json` — Field schemas per block and page metadata
- `component-filters.json` — Allowed blocks per section context

Custom page metadata fields extend the boilerplate with `target`, `analytics`, `adventureCategory`, and `journeyStage` for martech audience segmentation.

Custom block models include: `accordion-faq-item`, `cards-steps-item`, `carousel-blog`, `carousel-blog-post`, `carousel-hero-slide`, `hero-adventure`, `marquee-ticker-item`, `tabs-activity-tab`, `youtube-video`, and column variants.

Templates

Template	Purpose	Applied Via	Special Behavior	Figma
landing-page	Marketing landing pages	template: landing-page	Full marketing page layout	Hi-Fi
blog-article	Long-form field notes / blog posts	template: blog-article in metadata	Loads <code>styles/blog.css</code> ; body class <code>blog-article</code> ; hero-adventure + article columns	Hi-Fi
field-notes	Field notes listing hub	template: field-notes	Listing page structure for / field-notes	Hi-Fi

Template HTML previews live in `templates/{name}/{name}.html` for library and local testing. `scripts.js` applies template as body classes via `applyTemplateAndTheme()`.

Template overview: Templates — Hi-Fi

GitHub Workflows

Workflow	Trigger	Purpose
main.yaml	Push	Runs npm ci + npm run lint on Node 24
daily-traffic-simulation.yaml	Cron 06:00 UTC + manual	Simulates ~10k Analytics hits against production
cleanup-on-create.yaml	Repository events	Boilerplate cleanup on repo creation

The daily traffic workflow persists a visitor-pool artifact so virtual visitors retain ECIDs across runs, producing realistic returning-visitor Analytics data.

Tools and Traffic Simulator

Located in tools/scripts/:

Script	Purpose
simulate-live-audience-traffic.mjs	Playwright-based production traffic simulator
batch-traffic-backfill.mjs	Multi-day backfill wrapper (unique seed per day)
lib/audience-traffic-profiles.mjs	WKND audience persona definitions
lib/site-index.mjs	Fetches query-index.json from production
lib/traffic-visitors.mjs	Visitor pool, storage state, schedule builder
lib/enrich-profiles-from-index.mjs	Maps index pages to audience profiles

How it works:

1. Refreshes query-index.json from <https://main-masterclass-demo-znikolovski.aem.live>
2. Builds a randomized audience mix from WKND persona profiles
3. Launches Playwright browsers with persisted storage per virtual visitor
4. Navigates clustered page paths, waits for Web SDK hits (~3.5s)
5. Saves visitor pool to tools/scripts/output/visitor-pool/ for cross-run ECID reuse

Quick local dry run

```
npm run simulate:traffic:dry
```

Full daily volume (matches CI)

```
npm run simulate:traffic:daily
```

Multi-day backfill

```
npm run simulate:traffic:backfill
```

Common Development Tasks

Add a New Block

1. Create blocks/{name}/{name}.js with export default async function decorate(block)
2. Create blocks/{name}/{name}.css with scoped selectors (. {name} .element)
3. Add model entries to component-models.json and palette entry to component-definition.json
4. Add a hi-fi component frame to Blocks — Hi-Fi and update project-guides/.figma-reference.json with hifiNodeId

5. Test locally at `http://localhost:3000`
6. Push to a feature branch and verify on `https://{branch}--masterclass-demo--znikolovski.aem.page/`

Modify Global Styles

1. Update tokens in Figma Foundations and mirror changes in `styles/brand.css`
2. Edit `styles/styles.css` for layout and section styles
3. Test across homepage, blog article, and landing page templates
4. Run PageSpeed Insights on preview URL — watch for CLS impact

Add Analytics or Marketing Integration

1. Add Launch rules in the Adobe Tags property (see `docs/ANALYTICS-LAUNCH-PLAN.md`)
2. For block interactions, use `pushInteractionEvent()` from `analytics-acdl.js`
3. For page-level data, extend `buildPageContext()` in `analytics-page.js`
4. Never add tracking scripts to `scripts.js` — use martech phased hooks or `delayed.js`

Debug a Block

1. Check browser console for decoration errors
2. Compare expected block table structure vs delivered HTML (`curl http://localhost:3000/path.plain.html`)
3. Cross-check the hi-fi spec cell structure on Blocks — Hi-Fi against delivered markup
4. Verify variant/section classes are applied via `section-metadata`
5. Check Network tab for 404s on block JS/CSS

Environments

Environment	URL Pattern	Purpose
Local	<code>http://localhost:3000</code>	Development
Feature Branch	<code>https://{branch}--masterclass-demo--znikolovski.aem.page/</code>	PR testing
Preview (main)	<code>https://main--masterclass-demo--znikolovski.aem.page/</code>	Staging / author preview
Live (main)	<code>https://main--masterclass-demo--znikolovski.aem.live/</code>	Production

Git Workflow

Branch Naming

- Features: `feature/description`
- Fixes: `fix/description`
- Keep names short (URL length limits affect preview URLs)

PR Requirements

1. Include a preview URL in the PR description (required for review)
 2. Pass `npm run lint` (enforced by CI)
 3. Test on feature branch preview environment
 4. Run PageSpeed Insights on changed pages — target score 100
 5. No console errors in browser DevTools
-

Troubleshooting

Block not loading?

1. Verify block folder name matches the class on the block div
2. Confirm JS exports default function `decorate`
3. Check Network tab for 404 on `{block}.js` or `{block}.css`

Styles not applying?

1. Check CSS specificity — all selectors must be scoped to the block
2. Verify the CSS file loads in Network tab
3. Run `npm run lint:css` for syntax errors
4. Compare implementation against Figma Hi-Fi spec

Content not updating locally?

1. Hard-refresh browser (Cmd+Shift+R)
2. Confirm content was previewed/published in Document Authoring
3. Wait 1–2 minutes for CDN cache on remote previews

Martech not firing?

1. Confirm `martech-config.js` has real `datastream/org` IDs (not `REPLACE` placeholders)
2. Check page metadata — `analytics: off` disables page views
3. On localhost, staging Launch URL loads; verify in Network tab
4. Run `tools/scripts/test-analytics-acdl.mjs` for ACDL validation

Traffic simulator failures?

1. Install Playwright: `npx playwright install chromium`
 2. Ensure production site is reachable
 3. Use `--dry-run` to inspect schedule without browser launch
 4. Check `tools/scripts/output/traffic-run-*.json` for error details
-

Resources

Resource	URL
Edge Delivery Services Docs	https://www.aem.live/docs/
Developer Tutorial	https://www.aem.live/developer/tutorial
Block Collection	https://www.aem.live/developer/block-collection
E-L-D Loading	https://www.aem.live/developer/keeping-it-100
Martech Integration	https://www.aem.live/developer/martech-integration
Config Service API	https://www.aem.live/docs/admin.html
Best Practices	https://www.aem.live/docs/dev-collab-and-good-practices
Figma Design System	https://www.figma.com/design/6cx6VFkkgGJqijhoROnYDE/WKND-Adventures—EDS-Design-System

Support Contacts

Role	Contact
Repository Owner	znikolovski (GitHub)
Adobe Analytics Report Suite	ags050wknd (see docs/ANALYTICS-LAUNCH-PLAN.md)